Day 4 — High-Level Technical Modeling

> **Activity packet** for participant triads. Today's job: tie data, cloud, and API together into a **sequence diagram** that names every hop, protocol, and failure handler. Plus 1–2 sad/weird-path sequences. This is TMD Section 4.

Where we are in the week

Days 1–3 produced TMD Section 1 (data), Section 2 (cloud), Section 3 (API). Today integrates them: we draw the **system in motion**. A sequence diagram for the happy path; abbreviated sequences for the most important sad and weird paths.

Tomorrow's performance baselines and monitoring plan reference today's diagrams.

Inputs

- TMD Section 1 (entities)
- TMD Section 2 (cloud topology)
- TMD Section 3 (API contract)
- TCD Section 2 (Container diagram)
- TCD Section 4 (SLOs — drives latency annotations on the diagram)

Sequence diagrams — the right diagram for the right job

A sequence diagram shows **time flowing top to bottom** and **actors as vertical lifelines**. Messages between lifelines are arrows, ordered by time.

It is the right diagram for:

- A specific user action's journey through the system
- The interaction between services / modules during a transaction
- The order in which things happen — and where they could go wrong

It is the wrong diagram for:

- Showing all possible interactions (use a Container or Component diagram)
- Showing data structure (use an entity-relationship diagram)
- Showing deployment topology (use the cloud topology from Section 2)

What goes on the diagram

Element	What it represents
Lifeline (vertical line)	An actor — user, mobile app, service, datastore, queue
Arrow	A message — usually labeled with the operation + protocol
Solid arrow	Synchronous call (caller waits)
Dashed arrow	Async / fire-and-forget
Return arrow	Response (often dotted)
Note	Latency budget, condition, or important annotation
Alt / opt block	Conditional branch ("if 4xx", "if cache hit")

A good sequence diagram has **5–12 lifelines** (more becomes unreadable) and 10–25 arrows.

The three sequences every TMD Section 4 needs

1. **Happy path** — the central success case
2. **One sad path** — the most common user error or system error you have to handle gracefully
3. **One weird path** — an edge case from your AC's "weird" coverage (network drop, race, timeout, boundary)

Sad and weird paths can be sketched more abbreviated than the happy path — they often diverge from the happy path at a single arrow.

Activity 1 — Happy-Path Sequence

Format: Triad • **35 min** • Block 1

Purpose

Draw the central happy-path sequence diagram. Every lifeline, every arrow, every protocol.

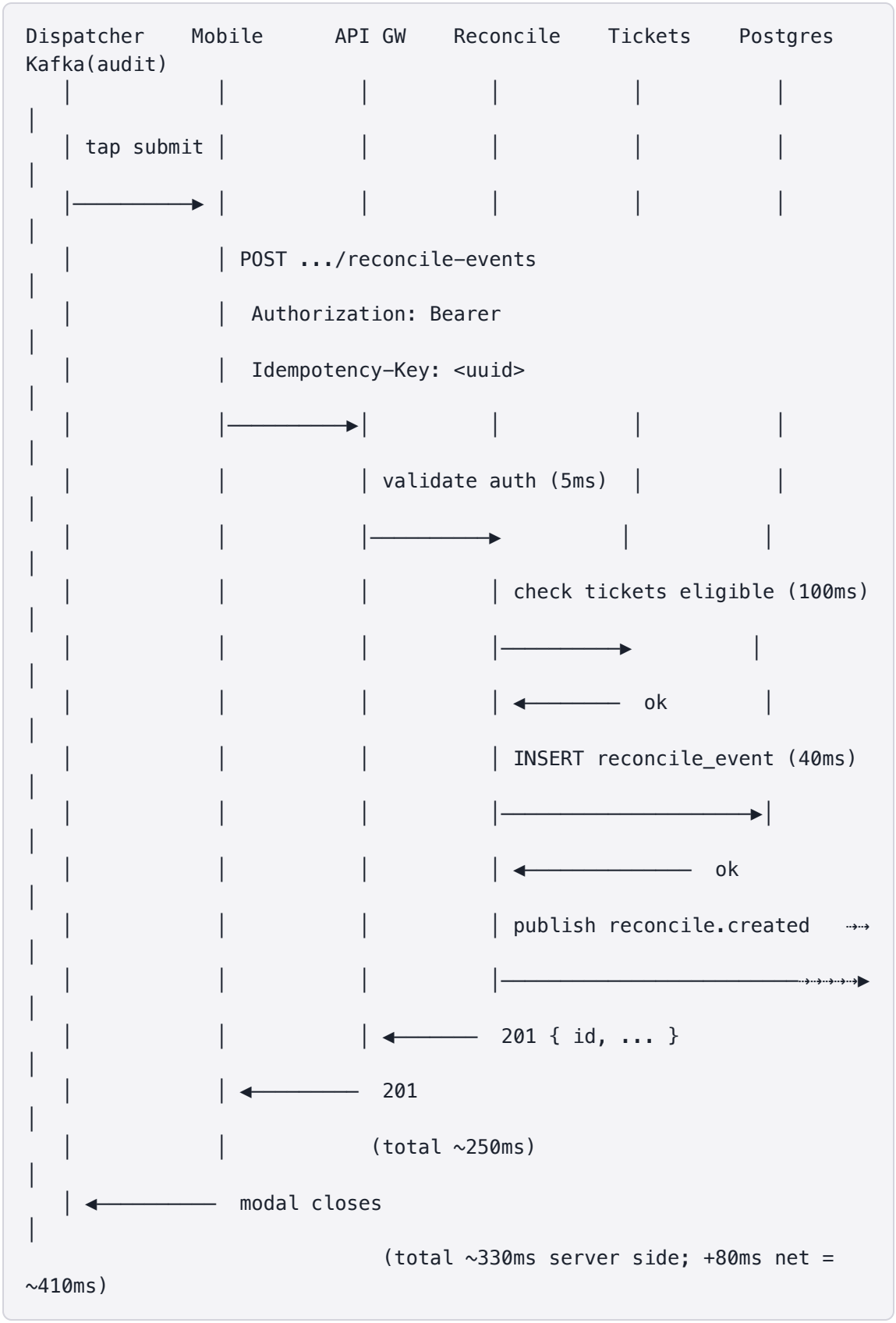
Setup

Each triad needs TCD Section 2 (Container diagram), TMD Sections 1–3, the TCD Section 4 latency-budget walk, and whiteboard or paper. AI optional.

Triad protocol

1. **Identify the actors** (5 min). Pull from TCD Section 2 + TMD Section 2 + Section 3. Examples for FieldPulse reconcile:
    - Dispatcher (user)
    - Mobile app
    - API Gateway
    - Reconcile module
    - Tickets module
    - Postgres
    - Audit topic (Kafka)
2. **Order the lifelines** (5 min). Caller on the left; called on the right.
3. **Draw the messages** (15 min). Solid for sync, dashed for async. Label with method + path.
4. **Annotate latency budget per arrow** (5 min). Pull from yesterday's TCD Section 4 latency-budget walk.
5. **The 60-second test** (5 min). Walk the diagram aloud to another triad in 60 seconds.

Worked example — FieldPulse reconcile happy path



What "good" looks like

- Every arrow has a protocol label
- Every synchronous arrow has a latency annotation
- The diagram fits one page (or one whiteboard)
- The total latency at the bottom **matches the SLO budget**

Deliverable

A happy-path sequence diagram with labeled lifelines, protocols, per-arrow latency annotations, and a total that matches the TCD Section 4 SLO.

Activity 2 — Sad-Path Sequence

Format: Triad • **40 min** • Block 2

Purpose

Pick the most consequential sad path from your AC — usually a user-error scenario — and draw the sequence.

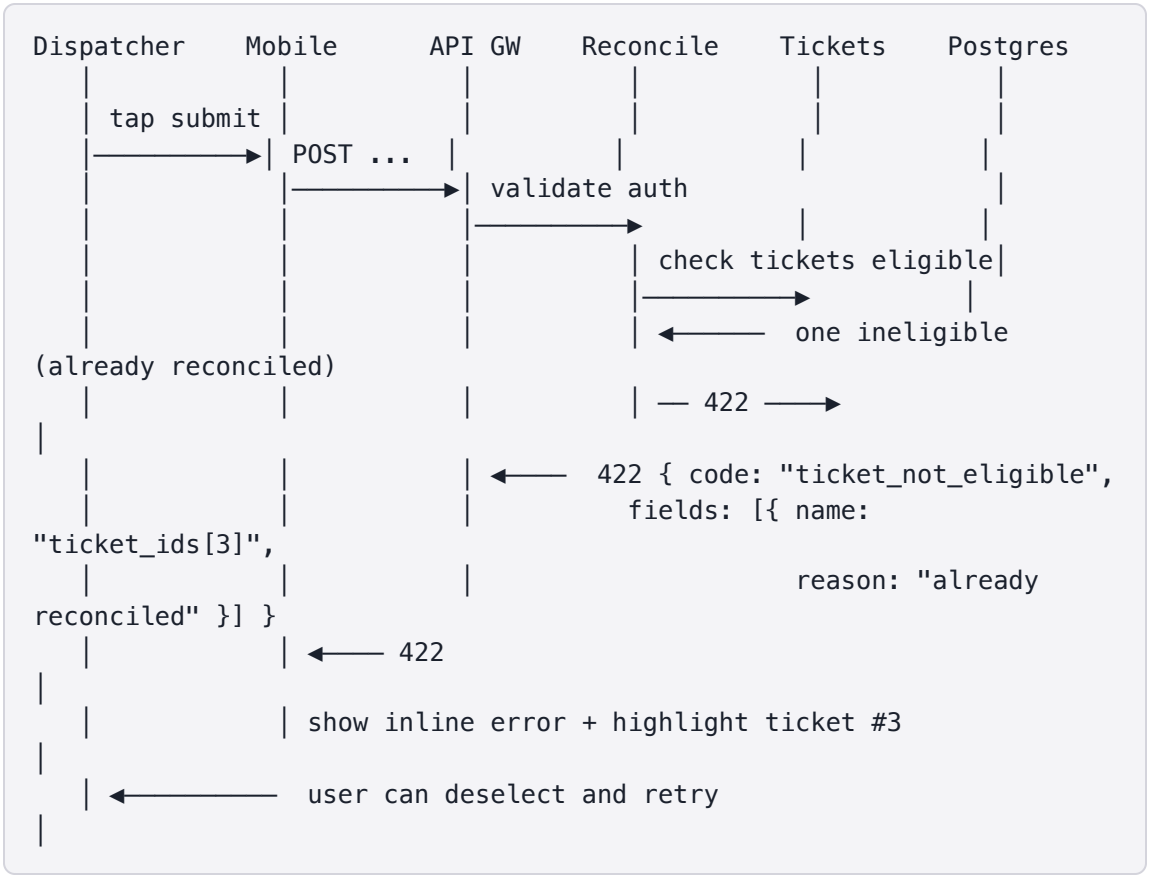
Setup

Each triad needs PRD Section 6 sad-path AC, TMD Section 3 (API error codes), and the happy-path sequence from Activity 1.

Triad protocol

1. **Pick the sad path** (5 min). From PRD Section 6 sad-path AC. Examples: "user submits with no tickets selected"; "user lacks permission"; "ticket already reconciled by someone else".
2. **Draw the divergence** (20 min). The sad path usually shares the first 1–2 arrows with the happy path, then branches. Draw from the divergence point.
3. **Show the user-visible result** (5 min). What does the user see? What status code? What error body?
4. **Annotate** (5 min). What's the user's recovery action?
5. **The 30-second walk** (5 min). To another triad.

Worked example — sad path: ticket already reconciled



What "good" looks like

- The divergence point is clear (one arrow that goes differently)
- The status code matches the API contract from Section 3
- The user has a **recovery path** — not a dead-end

Deliverable

A sad-path sequence diagram with the divergence point marked, the user-visible error code/body, and a named recovery action.

Activity 3 — Weird-Path Sequence

Format: Triad • **40 min** • Block 3

Purpose

Draw a weird-path sequence — network drop, race, timeout, or boundary case.

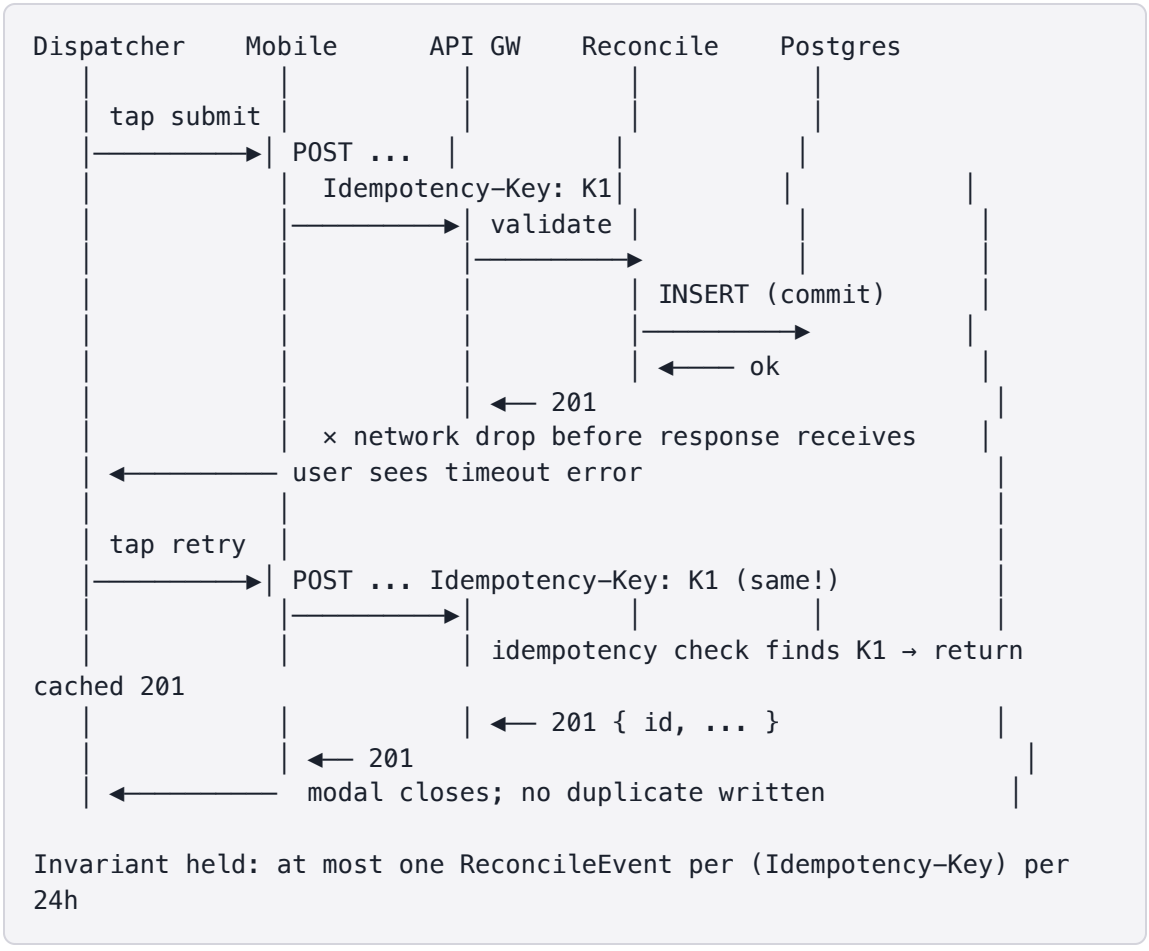
Setup

Each triad needs PRD Section 6 weird-path AC, TMD Section 3 (idempotency approach), and the happy-path diagram for reference.

Triad protocol

1. **Pick the weird path** (5 min). From PRD Section 6 weird-path AC.
    - Network drop mid-submit
    - Race: two dispatchers reconcile the same ticket simultaneously
    - Timeout: Tickets module slow
    - Boundary: 47 tickets when modal can list 25
2. **Draw the divergence + recovery** (25 min). Weird paths often need to show retry, idempotency, or fallback behavior.
3. **Annotate the system invariants that hold** (5 min). What can the user trust to be true even in this case?
4. **Walk it** (5 min) to another triad.

Worked example — weird path: network drop mid-submit



What "good" looks like

- The recovery is visible — what happens, who acts, what the system returns
- An invariant is named — the **promise we keep** even in this weird case
- The diagram is **shorter** than the happy path — focus on the divergence

Deliverable

A weird-path sequence diagram with a named invariant, visible recovery, and the idempotency or retry mechanism it relies on.

Activity 4 — Cross-Review + AI Sequence Critique

Format: Triad-pair • **45 min** + Wrap • Block 4

Purpose

Cross-review the three sequences with another triad. Then use AI to surface what was missed.

Setup

Instructor pairs triads. Each pair needs all three sequence diagrams in text form (for AI input) plus the cross-review prompt. AI required for the critique.

Cross-review protocol (20 min)

The reviewer triad reads the three sequences with these questions:

1. **Happy path:** does the latency add up? Are protocols consistent with Section 3?
2. **Sad path:** does the user have a clear recovery action?
3. **Weird path:** is an invariant named? Is the recovery actually safe?

The author triad listens, captures, decides what to address.

AI critique prompt

```
Role: Engineering reviewer with strong sequence-diagram instincts.
Context: <plain-text descriptions of the three sequence diagrams,
         including all lifelines, arrows, protocols, latencies,
         and annotations>
Task: Identify what's missing, ordered by likelihood of mattering
      in production:
  1. What sad/weird paths are NOT yet diagrammed but probably
     need to be?
  2. What arrow is missing from the happy path?
  3. What invariant should be added to the weird path that isn't?
Constraints:
  - Do not invent technologies; only use what's in the diagrams
  - Be specific
Format: Three sections; ranked items in each.
```

Triad action (15 min)

Adopt / defer / reject. Update the diagrams. Provenance note.

Deliverable

Updated TMD Section 4 with all three sequences, adopted cross-review and AI findings, and a provenance note for AI prompts.

Wrap (last 15 min)

Each triad shares:

- The **invariant** they're proudest of (from the weird path)
- The **arrow** AI surfaced as missing
- One question the diagrams couldn't answer

End-of-day checkpoint

Each triad ends Day 4 with:

- [x] **Happy-path sequence** with lifelines, protocols, latencies
- [x] **Sad-path sequence** with status code, error body, recovery
- [x] **Weird-path sequence** with invariant + recovery
- [x] Cross-reviewed with another triad
- [x] AI provenance log entry

- [x] TMD Section 4 drafted (with all three sequence diagrams)